

СЕМИНАР 2 · ВВЕДЕНИЕ В БАЗЫ ДАННЫХ

Идентичность и ключи

Что значит «тот же самый объект»?

Схема аукциона: таблица User

User	
<u>user_id</u>	BIGINT PK
email	VARCHAR NN, UNIQUE
phone	VARCHAR
display_name	VARCHAR NN
registered_at	TIMESTAMPTZ NN

ВОПРОС

email уникален.

**Почему бы не сделать его
первичным ключом?**

Какой набор атрибутов однозначно определяет строку?

```
Room(building_no, room_no, floor, capacity)
```

ВОПРОС

Аудитории всех корпусов университета.

Какой здесь ключ — и **нужны ли в нём floor и capacity?**

Какой набор атрибутов однозначно определяет строку?

```
Room (building_no, room_no, floor, capacity)
```

ОТВЕТ

(building_no, room_no) — аудитория X есть в каждом корпусе, а floor и capacity для однозначности не нужны.

ВЫВОД

Ключ **составной**. Это слабая сущность с лекции 2: номер уникален только внутри здания.

Какой набор атрибутов однозначно определяет строку?

```
LotTag (lot_id, tag_id)
```

ВОПРОС

Таблица из схемы аукциона.

Какой здесь ключ — и **что он запрещает в реальном мире?**

Какой набор атрибутов однозначно определяет строку?

```
LotTag (lot_id, tag_id)
```

ОТВЕТ

(lot_id, tag_id) — нельзя повесить на лот один и тот же тег дважды.

ВЫВОД

Любой ключ — это **запрет**. Модель описывает, какие состояния мира допустимы.

Какой набор атрибутов однозначно определяет строку?

```
Enrollment(student_id, course_id, semester, grade)
```

ВОПРОС

Предложите, какой набор атрибутов здесь будет ключом

Какой набор атрибутов однозначно определяет строку?

```
Enrollment(student_id, course_id, semester, grade)
```

ВОПРОС

Предложите, какой набор атрибутов здесь будет ключом

А ЧТО ЕСЛИ...

Студент завалил курс и пересдаёт его в следующем семестре.

Какой набор атрибутов однозначно определяет строку?

```
Enrollment(student_id, course_id, semester, grade)
```

ОТВЕТ

Нельзя пересдавать: **(student_id, course_id)**

Можно пересдавать: **(student_id, course_id, semester)**

ТЕЗИС 1

Ключ — это утверждение о мире, а не о данных.

То, что сейчас в колонке нет повторов, ещё не делает её ключом.

От суперключа к первичному ключу

```
Student(student_id, email, passport, full_name, birth_date)
```

Термин	Что это	Пример
Суперключ	Любой набор атрибутов, однозначно определяющий строку	{student_id}, {student_id, full_name}
Кандидатный ключ	Минимальный суперключ: ни один атрибут нельзя убрать	{student_id}, {email}, {passport}
Первичный ключ (PK)	Кандидатный ключ, выбранный для ссылок из других таблиц	student_id
Альтернативный ключ	Кандидатный ключ, который не стал первичным → UNIQUE	email, passport

Это ключ?

```
Student(student_id, email, passport, full_name, birth_date)
```

ВОПРОС

1. {student_id, full_name} — ключ?
2. full_name — кандидатный ключ?
3. (full_name, birth_date) — кандидатный ключ?

Это ключ?

```
Student(student_id, email, passport, full_name, birth_date)
```

ОТВЕТ

1. **Суперключ**, но не кандидатный: full_name можно убрать
2. **Нет**: тёзки существуют
3. **Нет**: тёзки с одной датой рождения тоже существуют

Сколько здесь клиентов?

Сервис записи на занятия. Администратор выгрузил список клиентов:

#	name	phone	email
1	Иван Петров	+7 916 123-45-67	ivan@gmail.com
2	И. Петров	89161234567	ivan@gmail.com
3	Иван Петров	+7 916 123-45-67	NULL
4	Ivan Petrov	NULL	ivan@gmail.com
5	Иван Петров	+7 916 777-88-99	ivan2@gmail.com

ВОПРОС

Сколько здесь клиентов? Предложите свой ответ — число от 1 до 5

Имя и телефон совпадают, почты нет

#	name	phone	email
1	Иван Петров	+7 916 123-45-67	ivan@gmail.com
2	И. Петров	89161234567	ivan@gmail.com
3	Иван Петров	+7 916 123-45-67	NULL
4	Ivan Petrov	NULL	ivan@gmail.com
5	Иван Петров	+7 916 777-88-99	ivan2@gmail.com

ВОПРОС

Что должно быть правдой, чтобы это был один человек? А чтобы разные?

Имя и телефон совпадают, почты нет

#	name	phone	email
1	Иван Петров	+7 916 123-45-67	ivan@gmail.com
2	И. Петров	89161234567	ivan@gmail.com
3	Иван Петров	+7 916 123-45-67	NULL
4	Ivan Petrov	NULL	ivan@gmail.com
5	Иван Петров	+7 916 777-88-99	ivan2@gmail.com

ВОПРОС

Что должно быть правдой, чтобы это был один человек? А чтобы разные?

А ЧТО ЕСЛИ...

Отец и сын — оба Иваны Петровы. Отец записал сына на занятие **со своего телефона**.

Имя и телефон совпадают, почты нет

#	name	phone	email
1	Иван Петров	+7 916 123-45-67	ivan@gmail.com
2	И. Петров	89161234567	ivan@gmail.com
3	Иван Петров	+7 916 123-45-67	NULL
4	Ivan Petrov	NULL	ivan@gmail.com
5	Иван Петров	+7 916 777-88-99	ivan2@gmail.com

ВОПРОС

Что должно быть правдой, чтобы это был один человек? А чтобы разные?

ОТВЕТ

NULL — не «почты нет», а «почта неизвестна».

Кто клиент: тот, кто ходит на занятия, или тот, кто платит?

Тот же человек сменил контакты — или однофамилец?

#	name	phone	email
1	Иван Петров	+7 916 123-45-67	ivan@gmail.com
2	И. Петров	89161234567	ivan@gmail.com
3	Иван Петров	+7 916 123-45-67	NULL
4	Ivan Petrov	NULL	ivan@gmail.com
5	Иван Петров	+7 916 777-88-99	ivan2@gmail.com

ВОПРОС

Один это человек или два? **Чего не хватает, чтобы ответить?**

Тот же человек сменил контакты — или однофамилец?

#	name	phone	email
1	Иван Петров	+7 916 123-45-67	ivan@gmail.com
2	И. Петров	89161234567	ivan@gmail.com
3	Иван Петров	+7 916 123-45-67	NULL
4	Ivan Petrov	NULL	ivan@gmail.com
5	Иван Петров	+7 916 777-88-99	ivan2@gmail.com

ВОПРОС

Один это человек или два? **Чего не хватает, чтобы ответить?**

ОТВЕТ

Правильный ответ — **«не знаю»**.

Нужны: дата рождения, даты создания записей, подтверждён ли телефон/почта, источник записи.

Ответ — не число, а правило

Политика идентификации — например:

1. Клиент — **тот, кто посещает занятие**; плательщика храним отдельно
2. Телефоны и почты сравниваем **после нормализации**
3. Совпал **подтверждённый** телефон или **подтверждённая** почта — один клиент
4. Совпало только имя — **не основание** объединять
5. Спорные случаи **не склеиваем автоматически**, а отдаём на проверку администратору

Две разные вещи

Идентичность

Решаем, что значит «тот же клиент».

Это **решение бизнеса**, принимается до проектирования таблиц.

Дедупликация

Ищем записи, которые по этому правилу об одном клиенте.

Это **процедура**, чистка уже накопившихся дублей.

ТЕЗИС 2

Сначала идентичность, потом дедупликация.

Чем идентифицировать клиента?

Свойство	Вопрос
Уникальность	Может ли одно значение быть у двух разных клиентов?
Стабильность	Меняется ли значение у одного клиента со временем?
Обязательность	Есть ли это значение у каждого клиента при регистрации?
Чувствительность	Насколько опасна утечка? Нужно ли это вообще хранить?
Исправимость	Что будет, если значение ввели с ошибкой и его надо поправить?

Заполняем вместе

	Уникальность	Стабильность	Обязательность	Чувствит.	Исправимость
--	--------------	--------------	----------------	-----------	--------------

phone

email

telegram_id

passport

customer_id

ВОПРОС

Давайте пройдем по строкам. Предложите **плюс или минус** для каждого свойства

Что получилось

	Уникальность	Стабильность	Обязательность	Чувствит.	Исправимость
phone	нет: общий семейный номер	нет: меняют; номера переиздают	не всегда	персональные данные	правка тянет ссылки
email	почти: бывают общие ящики	средне: меняют	нет: вход по телефону	персональные данные	правка тянет ссылки
telegram_id	да	да: id не меняется, username меняется	нет: Telegram не у всех	средняя	нельзя: внешнее значение
passport	да	нет: 20 и 45 лет, утеря	нет: зачем для записи?	очень высокая	опечатки
customer_id	да, по построению	да	да, выдаём сами	ничего не раскрывает	не требуется

Ни один естественный идентификатор не проходит все пять проверок.

Берём суррогатный customer_id. Всё решено?

<u>customer id (PK)</u>	name	phone	email
1	Иван Петров	+79161234567	ivan@gmail.com
2	Иван Петров	+79161234567	ivan@gmail.com

ВОПРОС

Нарушен ли первичный ключ?

ТЕЗИС 3

Суррогатный ключ ничего не знает о дубликатах.

От дублей защищает UNIQUE на естественных атрибутах.

UNIQUE — это бизнес-правило

ВОПРОС

Ставим **UNIQUE** на email? А на телефон?

EMAIL

UNIQUE(email) запрещает семье общий ящик.

Это надо согласовать с заказчиком.

ТЕЛЕФОН

UNIQUE(phone) — родитель записывает двоих детей со своего номера.

Почти наверняка ошибка.

UNIQUE = предложение «в нашем мире у двух клиентов не бывает одинаковой почты».
В PostgreSQL UNIQUE по умолчанию допускает сколько угодно NULL.

Автомобиль

Идентификаторы:

заводской номер машины (VIN) · госномер · внутренний id

ВОПРОС

1. Что здесь **естественный ключ**?
2. Может ли он **измениться** или быть **неизвестен**?
3. Почему **техническая ссылка** может быть другой?

Автомобиль

Идентификатор	Что с ним не так
Госномер	Идентификатор регистрации , а не машины: при продаже номера меняют, старый номер выдают другой машине
Заводской номер (VIN)	Хороший естественный ключ: не меняется за жизнь машины. Но когда клиент записывается на ремонт по телефону, номер неизвестен , а ещё его вводят с ошибками
Внутренний id	Стабильная ссылка для записей на ремонт, даже если заводской номер пришлось исправлять

ВЫВОД

car_id — для ссылок, **UNIQUE(vin)** — заводской номер не повторяется, госномер — обычный атрибут

Товар на маркетплейсе

Один и тот же iPhone продают **десять продавцов** — у каждого своя цена

ВОПРОС

Это один товар или десять?

Что здесь должно получить свой идентификатор?

Товар на маркетплейсе

```
Product(product_id, name)
Offer(offer_id, product_id, seller_id, price)
```

ОТВЕТ

И то и другое. iPhone в каталоге — **один товар**. Десять продавцов с разной ценой — **десять предложений**.

ВЫВОД

Прежде чем выбирать ключ, решаем, **что мы называем товаром**. Здесь это две сущности — у каждой свой id.

Телефон как внешний ключ

```
Customer(phone PK, name, email)
Order(order_id PK, customer_phone FK, created_at, amount)
customer_phone → Customer.phone
```

ВОПРОС

Клиент сменил номер. Что произойдёт?

Что сломается

```
Customer(phone PK, name, email)
Order(order_id PK, customer_phone FK, created_at, amount)
customer_phone → Customer.phone
```

1. Меняем телефон — ломаются
ссылки: заказы указывают на номер,
которого больше нет

2. Заводим клиента с новым номером
— у человека два аккаунта, история
разорвана

САМОЕ НЕПРИЯТНОЕ

3. Оператор **выдал старый номер другому человеку**. Он регистрируется —
и к нему «приклеиваются» чужие заказы.

Как это исправить? Предложите хотя бы два способа

Исправление	Что решает	Чего не решает
-------------	------------	----------------

Суррогатный `customer_id`, заказ
ссылается на него

Телефон на момент заказа в
`Order` — обычное поле, не FK

Таблица телефонов клиента с
датами действия

ON UPDATE CASCADE

Как это исправить?

Исправление	Что решает	Чего не решает
Суррогатный customer_id, заказ ссылается на него	Смена номера — одна строка, заказы не трогаем	Дубли клиентов — нужна политика и UNIQUE
Телефон на момент заказа в Order — обычное поле, не FK		
Таблица телефонов клиента с датами действия		
ON UPDATE CASCADE		

Как это исправить?

Исправление	Что решает	Чего не решает
Суррогатный customer_id, заказ ссылается на него	Смена номера — одна строка, заказы не трогаем	Дубли клиентов — нужна политика и UNIQUE
Телефон на момент заказа в Order — обычное поле, не FK	Видно, с какого номера был заказ	Сам по себе не связывает заказ с клиентом
Таблица телефонов клиента с датами действия		
ON UPDATE CASCADE		

Как это исправить?

Исправление	Что решает	Чего не решает
Суррогатный <code>customer_id</code> , заказ ссылается на него	Смена номера — одна строка, заказы не трогаем	Дубли клиентов — нужна политика и <code>UNIQUE</code>
Телефон на момент заказа в <code>Order</code> — обычное поле, не <code>FK</code>	Видно, с какого номера был заказ	Сам по себе не связывает заказ с клиентом
Таблица телефонов клиента с датами действия	История номеров	Сложнее — только если это спрашивают
ON UPDATE CASCADE		

Как это исправить?

Исправление	Что решает	Чего не решает
Суррогатный <code>customer_id</code> , заказ ссылается на него	Смена номера — одна строка, заказы не трогаем	Дубли клиентов — нужна политика и <code>UNIQUE</code>
Телефон на момент заказа в <code>Order</code> — обычное поле, не <code>FK</code>	Видно, с какого номера был заказ	Сам по себе не связывает заказ с клиентом
Таблица телефонов клиента с датами действия	История номеров	Сложнее — только если это спрашивают
<code>ON UPDATE CASCADE</code>	Смена номера обновит заказы	Не решает переиздание номера, переписывает историю

Итоговый вопрос

ВОПРОС

Почему суррогатный PRIMARY KEY не гарантирует отсутствие двух записей об одном клиенте?

Попробуйте ответить в одном-двух предложениях.

Как менялся ключ одной таблицы

Таблица **managed_vips** рабочего сервиса, который выдаёт командам адреса для балансировщиков нагрузки. Одна строка — адрес, порт, протокол и кластер.

```
PRIMARY KEY (address, port, protocol, cluster)
```

ЗАЧЕМ МЫ НА НЕЁ СМОТРИМ

Сначала ключ был естественным. Дальше — **три решения** из жизни этой схемы, и каждое по правилам, которые мы сегодня разобрали.

Адрес выдаётся позже, чем создаётся запись

ЧТО ИЗМЕНИЛОСЬ

Запись создаётся **операцией** (таблица operations), а адрес выделяется потом. Пока адреса нет, его колонка — NULL. А **первичный ключ не может быть NULL**.

```
id uuid PRIMARY KEY
UNIQUE INDEX (address, port, protocol, cluster)
```

РЕШЕНИЕ

Первичный ключ — **суррогатный id**. Естественный ключ остался **бизнес-правилом** в виде уникального индекса. Как с заводским номером машины: при создании его ещё нет.

Удалённые записи мешают новым

ЧТО ИЗМЕНИЛОСЬ

Записи не удаляют, а помечают **is_deleted**. Адрес освободился и выдаётся новому сервису — но уникальный индекс видит старую удалённую запись и не пускает новую.

```
        UNIQUE INDEX (address, port, protocol, cluster)
WHERE is_deleted = false
```

РЕШЕНИЕ

Уникальность **только среди активных** записей. Удалённых с тем же адресом может быть сколько угодно — это история.

Не больше одной операции за раз

ЧТО ИЗМЕНИЛОСЬ

Записи меняются через **операции** (таблица operations): создать, изменить, удалить. Правило: над одной записью **не могут одновременно выполняться две операции**.

```
operations(id, managed_vip_id, status, ...)  
UNIQUE INDEX (managed_vip_id, status)  
WHERE status = 'running'
```

РЕШЕНИЕ

Уникальность **только среди выполняющихся** операций: завершённых над записью сколько угодно, выполняющаяся — одна. Проверяет база, поэтому два одновременных запроса не пройдут оба.